

Introduction to Knowledge Discovery and Data Mining, and Python Basics

CIS 635 — Knowledge Discovery & Data Mining

Course Reading Material

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions regarding the use of this material, contact hasaka@gvsu.edu.

Please cite [1] this material when using or reproducing it.

Unit: Foundations of KDD and Python Programming

Contents

1	Learning Objectives	3
2	Introduction to Knowledge Discovery and Data Mining	3
2.1	What Is Knowledge Discovery in Databases?	3
2.2	Why KDD Matters	4
3	The KDD Process and Its Major Stages	5
3.1	The Classic KDD Process	5
3.2	CRISP-DM: An Industry-Standard Alternative	6
4	The Purpose and Importance of Each Stage: A Worked Example	8
5	Real-World Applications of KDD and Data Mining	10
5.1	Retail and Marketing: Predicting Behavior from Purchase Patterns	11
5.2	Finance: Real-Time Fraud Detection	11
5.3	Healthcare: Early Warning Systems	12
5.4	Other Domains	12
5.5	A Cautionary Note: Not Every Famous Story Is True	13
6	Python Basics: Variables, Data Types, and Operators	14
6.1	Variables and Basic Data Types	14
6.2	Arithmetic Operators	14
6.3	Comparison, Logical, and Assignment Operators	15
7	Control Flow: Conditional Statements and Loops	16
7.1	Conditional Statements	16
7.2	for Loops	17
7.3	while Loops	17
7.4	List Comprehensions	18

8	Functions in Python	18
8.1	Defining and Calling a Function	19
8.2	Parameters, Return Values, and Default Arguments	19
8.3	Returning Multiple Values	19
8.4	Function Composition	20
9	Basic Data Structures: Lists, Tuples, Dictionaries, and Sets	21
9.1	Lists: Ordered and Mutable	21
9.2	Tuples: Ordered and Immutable	21
9.3	Dictionaries: Key-Value Mappings	22
9.4	Sets: Unordered Collections of Unique Elements	22
10	Best Practices and Common Pitfalls	24
11	A Practical Mental Model	24
12	Capstone Exercise: An End-to-End Mini Walkthrough	25
13	Summary	26
14	Discussion Questions	27
15	Key Takeaways for CIS 635	27

1 Learning Objectives

After completing this reading, you should be able to:

- Define knowledge discovery in databases (KDD) and data mining, and explain how these terms relate to, and differ from, machine learning, statistics, and artificial intelligence more broadly.
- Describe the classic KDD process (selection, preprocessing, transformation, data mining, interpretation/evaluation) and explain the purpose of each stage.
- Compare the classic KDD process to CRISP-DM, an industry-standard process model, and explain why both are described as iterative rather than linear.
- Identify real-world applications of KDD and data mining across domains such as retail, finance, and healthcare, and critically evaluate commonly repeated anecdotes for accuracy.
- Write and read basic Python programs using variables, built-in data types, arithmetic/-comparison/logical/assignment operators, conditional statements, loops, functions, and list comprehensions.
- Choose an appropriate built-in Python data structure (list, tuple, dictionary, or set) for a given data-representation task, and justify the choice.
- Trace a small, messy raw dataset through all five stages of the KDD process using only plain Python, and interpret the resulting pattern with appropriate caveats.

2 Introduction to Knowledge Discovery and Data Mining

Every day, hospitals log patient vitals, retailers log purchases, farms log sensor readings, and banks log transactions. On its own, a log is just a record of what happened. **Knowledge discovery in databases (KDD)** is the field concerned with turning large collections of such records into knowledge that people can actually act on: which patients are at elevated risk, which products tend to sell together, which fields are under-irrigated, which transactions look fraudulent. This reading introduces the vocabulary and process models that structure that work, and reviews the Python programming foundations you will build on throughout this course.

2.1 What Is Knowledge Discovery in Databases?

The term **knowledge discovery in databases** was popularized in the early 1990s to describe the overall process of finding valid, novel, useful, and understandable patterns in data. [2] **Data mining** is, strictly speaking, only one stage within that larger process — the stage where an algorithm is actually applied to extract patterns — but in casual usage, and increasingly in job titles and course names, “data mining” and “KDD” are used almost interchangeably to refer to the whole endeavor. This reading follows the common convention of using “KDD” when emphasizing the end-to-end process, and “data mining” when emphasizing the pattern-extraction step itself, while flagging the distinction explicitly here so the terminology does not become a source of confusion later.

KDD sits at the intersection of several older fields, and it is worth being precise about how they differ, because you will see all of these terms used loosely in industry:

Field	Primary concern
Statistics	Quantifying uncertainty and testing hypotheses, typically starting from a question the analyst already has in mind and a model chosen in advance.
Databases	Efficiently storing, indexing, and querying large volumes of structured data, without necessarily analyzing it for patterns.
Machine learning (ML)	Algorithms that improve their performance on a task from data, typically to build predictive or generative models; a core toolbox used <i>during</i> the data mining stage of KDD.
Data mining	The specific KDD stage of applying an algorithm (often, but not always, a machine-learning algorithm) to extract patterns from prepared data.
Knowledge discovery in databases (KDD)	The full end-to-end process — selecting, cleaning, transforming, mining, and interpreting data — of which data mining is one stage.
Artificial intelligence (AI)	The broadest field, concerned with building systems that perform tasks associated with intelligence; machine learning (and, by extension, much of data mining) is one major approach within AI, not a synonym for it.

Key Idea

A useful way to keep these straight: statistics usually starts with a specific question and a chosen model; KDD usually starts with a pile of data and an open-ended goal of finding something useful in it. Machine learning supplies many of the algorithms KDD uses at the data-mining stage, but KDD is equally concerned with everything that has to happen *before* an algorithm can be applied at all — which, in practice, is most of the work.

2.2 Why KDD Matters

Organizations do not lack data; they lack *time and process* for turning data into decisions. A hospital’s electronic health record system already contains everything needed to notice that patients with a certain combination of vital signs tend to deteriorate — but nobody can review millions of records by hand. A KDD process gives that task structure: it specifies what gets selected, how it gets cleaned and reshaped, what algorithm is applied, and how the result gets checked before anyone acts on it. Skipping any of these steps, or performing them in an ad hoc order, is one of the most common reasons real data mining projects fail to produce anything useful, a theme this reading returns to throughout.

Practical Exercises

1. In your own words, explain the difference between “data mining” and “KDD.” Give an example of a task that belongs to KDD but is *not* the data mining stage itself.
2. Statistics, databases, machine learning, and data mining all work with data, but ask different questions of it. Pick a domain you are familiar with (school, work, a hobby) and

describe one question from that domain that a statistician would typically ask, and one that a data miner would typically ask, so that the difference in framing is clear.

3. Machine learning is sometimes described as “a subset of AI,” and data mining is sometimes described as “an application of machine learning and statistics.” Are these two statements consistent with each other? Explain why or why not, referring to the table above.
4. A small clinic wants to know “which patients are likely to miss their next appointment.” Is this more naturally framed as a data mining question or a database-reporting question? Justify your answer in two or three sentences.

3 The KDD Process and Its Major Stages

KDD is not a single algorithm; it is a *process* made up of several stages, most of which have nothing to do with running a machine-learning model. This section presents two ways of describing that process: the classic, academically-oriented KDD process, and CRISP-DM, the process model most widely used in industry today. Both describe essentially the same underlying work, organized somewhat differently.

3.1 The Classic KDD Process

The most widely cited formulation of the KDD process comes from Fayyad, Piatetsky-Shapiro, and Smyth (1996), who described it as five stages moving from raw data to knowledge: [2]

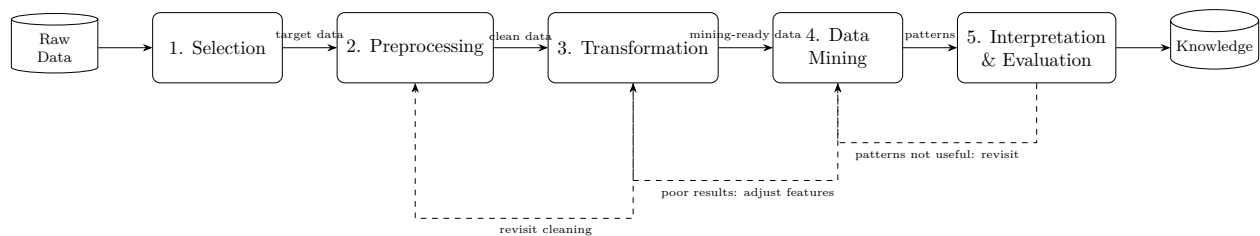


Figure 1: The classic KDD process. Raw data is narrowed to a target subset, cleaned, transformed into a form suitable for mining, mined for patterns, and finally interpreted and evaluated to produce actionable knowledge. In practice the process is rarely a single pass: any stage may reveal that an earlier stage needs to be repeated.

Briefly, each stage does the following:

1. **Selection.** Decide which data, and which parts of it, are actually relevant to the question at hand. A hospital’s full EHR system might contain hundreds of tables; a study of sepsis risk needs only a handful of them.
2. **Preprocessing.** Address data-quality problems: missing values, duplicate records, sensor errors, inconsistent formats, and outliers that reflect data-entry mistakes rather than real phenomena.
3. **Transformation.** Reshape the cleaned data into a form suitable for the mining algorithm: deriving new features, aggregating records, normalizing scales, or converting categorical labels into a numeric or coded representation.
4. **Data Mining.** Apply an algorithm — a clustering method, a classifier, an association-rule miner, a regression model — to extract candidate patterns from the transformed data.

5. **Interpretation and Evaluation.** Decide whether the patterns found are valid, novel, and actually useful, and translate them into a form domain experts and decision-makers can act on.

Key Idea

The dashed feedback arrows in Figure 1 are not an afterthought; they are the point. A pattern that turns out to be a data-quality artifact sends you back to Preprocessing. A model that performs poorly may need different features from Transformation. Treating KDD as a strict one-way pipeline is one of the most common misconceptions students bring into this course.

3.2 CRISP-DM: An Industry-Standard Alternative

While the Fayyad et al. process is the standard academic reference, most practicing data scientists describe their work using **CRISP-DM** (Cross-Industry Standard Process for Data Mining), a process model developed by an industry consortium in the late 1990s. [3] CRISP-DM organizes the same underlying work into six phases, explicitly framed around business goals rather than only the data itself:

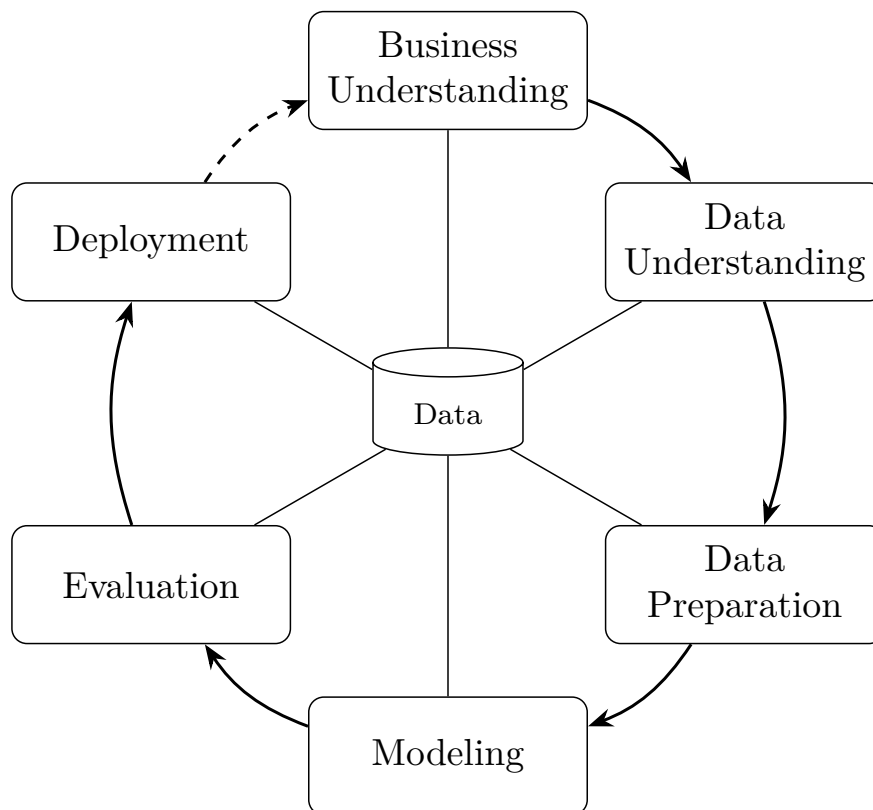


Figure 2: CRISP-DM (Cross-Industry Standard Process for Data Mining) as a cyclical, business-driven process. Each stage feeds a shared body of data and understanding, the outer arrows show the typical order of progression, and the dashed arrow marks the loop back to Business Understanding once a project is deployed and re-evaluated against business goals.

CRISP-DM's phases map fairly directly onto the classic KDD stages, with two additions that the academic formulation leaves implicit: an explicit *Business Understanding* phase at the start, and an

explicit *Deployment* phase at the end.

CRISP-DM phase	Nearest stage(s)	KDD	What it adds or emphasizes
Business Understanding	(implicit, before Selection)		Makes explicit that a project starts from a business objective and success criteria, not from a dataset.
Data Understanding	Selection		Includes exploratory profiling of the data before committing to what to select.
Data Preparation	Preprocessing, Transformation		Combines cleaning and reshaping into a single phase; in practice this is typically the most time-consuming phase of any project.
Modeling	Data Mining		Emphasizes trying and comparing multiple modeling techniques, not just applying one algorithm.
Evaluation	Interpretation & Evaluation		Evaluates the model against the original <i>business</i> objective, not only against a technical metric.
Deployment	(implicit, after Evaluation)		Makes explicit that a model must be put into production and monitored — a step the classic KDD process does not name separately.

Tip

When reading industry job postings and tooling documentation, you will encounter CRISP-DM far more often than the Fayyad et al. terminology. When reading academic papers on knowledge discovery, the reverse is often true. This course uses both because you will need to recognize both once you leave the classroom.

Practical Exercises

1. Redraw Figure 1 from memory (on paper or in a text list), labeling each arrow with what flows along it, before checking your answer against the figure.
2. CRISP-DM adds an explicit “Business Understanding” phase that the classic KDD process leaves implicit. Describe a scenario in which skipping this phase — diving straight into available data without first clarifying the business question — could lead a team to build a technically sound model that nobody actually needed.
3. Both process models are drawn with feedback loops rather than as a straight line. Using the farm-sensor or bookstore scenarios introduced later in this reading (or a scenario of your own), describe one concrete situation in which a data miner would need to go *backward* from Modeling/Data Mining to Data Preparation/Transformation.
4. Deployment is explicit in CRISP-DM but not named separately in the classic KDD process.

Does that mean the classic KDD process ignores deployment? Explain your reasoning, referring to where deployment-like concerns might fit into the five classic stages.

4 The Purpose and Importance of Each Stage: A Worked Example

The stage descriptions in Section 3 are easy to read and easy to underestimate. This section walks a small, deliberately messy dataset through all five classic KDD stages so that the *purpose* of each stage — and the consequences of skipping it — are concrete rather than abstract. The code below uses only plain Python (lists and dictionaries), previewed here and formalized in Sections 6–9; you do not need to understand every line yet to follow the reasoning.

Scenario. A small farm cooperative logs soil-moisture, rainfall, temperature, and crop-yield readings from field sensors. An agronomist wants to know: *does soil moisture relate to yield?* The raw export, however, has two of the data-quality problems every real KDD project encounters: a sensor that briefly went offline (recorded as a missing value), and a sensor fault code that looks like a number but is not a real reading.

```
raw_log = [
    {"field_id": "A1", "sensor_batch": 7, "soil_moisture_pct": 22.4,
     "rainfall_mm": 4.0, "temperature_c": 24.1, "yield_kg_per_ha": 3120},
    {"field_id": "A2", "sensor_batch": 7, "soil_moisture_pct": 18.1,
     "rainfall_mm": 1.2, "temperature_c": 26.5, "yield_kg_per_ha": 2870},
    {"field_id": "A3", "sensor_batch": 7, "soil_moisture_pct": None,      # sensor
     dropout
     "rainfall_mm": 0.0, "temperature_c": 27.8, "yield_kg_per_ha": 2510},
    {"field_id": "B1", "sensor_batch": 8, "soil_moisture_pct": 31.6,
     "rainfall_mm": 12.4, "temperature_c": 21.9, "yield_kg_per_ha": 3810},
    {"field_id": "B2", "sensor_batch": 8, "soil_moisture_pct": 29.0,
     "rainfall_mm": 9.8, "temperature_c": 22.4, "yield_kg_per_ha": 3655},
    {"field_id": "B3", "sensor_batch": 8, "soil_moisture_pct": -999.0,  # sensor fault
     code
     "rainfall_mm": 10.1, "temperature_c": 22.0, "yield_kg_per_ha": 3702},
    {"field_id": "C1", "sensor_batch": 9, "soil_moisture_pct": 15.2,
     "rainfall_mm": 0.4, "temperature_c": 29.3, "yield_kg_per_ha": 2255},
    {"field_id": "C2", "sensor_batch": 9, "soil_moisture_pct": 16.8,
     "rainfall_mm": 0.6, "temperature_c": 28.9, "yield_kg_per_ha": 2390},
]
```

Stage 1: Selection. The agronomist's question does not need `sensor_batch` or `temperature_c`, so we select only the fields relevant to it. This is not a minor bookkeeping step: carrying irrelevant fields forward makes every later stage slower and, in a real project with hundreds of columns, can actively mislead a mining algorithm that treats every column as potentially informative.

```
def select_fields(row):
    return {"field_id": row["field_id"],
           "soil_moisture_pct": row["soil_moisture_pct"],
           "yield_kg_per_ha": row["yield_kg_per_ha"]}

selected = [select_fields(row) for row in raw_log]
```


Stage 2: Preprocessing. Two problems are now visible: a missing reading (`None`) and an impossible reading (`-999.0`, a common convention for “sensor fault”). Neither should be treated as a real measurement.

```
cleaned = [row for row in selected
            if row["soil_moisture_pct"] is not None
            and row["soil_moisture_pct"] >= 0]
```

Removed 2 record(s) with missing or invalid readings.

Common Pitfall

If Preprocessing had been skipped, `-999.0` would have been averaged in as if a field were extremely dry, and `None` would have crashed any arithmetic performed on it. Both failure modes are common in real sensor and survey data, and both are silent unless someone deliberately checks for them — which is precisely what this stage is for.

Stage 3: Transformation. A raw percentage is harder for a simple rule to reason about than a category, so we derive a `moisture_category` label.

```
def categorize_moisture(pct):
    return "dry" if pct < 20.0 else "moist"

transformed = [{"row", "moisture_category": categorize_moisture(row["soil_moisture_pct"])
               }
               for row in cleaned]
```

Stage 4: Data Mining. With clean, transformed data in hand, the mining step itself is often the shortest part of the whole process. Here it is a simple group comparison: average yield, by moisture category.

```
def average(values):
    return sum(values) / len(values)

dry_yields = [r["yield_kg_per_ha"] for r in transformed if r["moisture_category"] == "dry"]
moist_yields = [r["yield_kg_per_ha"] for r in transformed if r["moisture_category"] == "moist"]
```

Average yield, dry fields: 2505.0 kg/ha (n=3)
Average yield, moist fields: 3528.3 kg/ha (n=3)

Stage 5: Interpretation and Evaluation. The numbers above are not, by themselves, knowledge. Interpretation means asking whether the pattern is real, generalizable, and useful enough to act on:

Moist fields out-yielded dry fields by 1023.3 kg/ha in this sample.
Caveat for the agronomist: n is tiny (8 fields, 2 sensor dropouts removed), rainfall and temperature were excluded at Selection, and no statistical test has been run -- this is a pattern worth investigating further, not yet a validated finding.

Key Idea

Notice what each stage actually contributed. Selection decided what mattered. Preprocessing prevented two silent errors from corrupting every later step. Transformation made the pattern easy to compute. Data Mining took three lines of code. Interpretation is where a human — not the code — decided the result was suggestive rather than conclusive. In most real KDD projects, that ordering of effort (a great deal on stages 1–3 and 5, comparatively little on stage 4 itself) is the norm, not the exception.

Practical Exercises

1. Suppose the Preprocessing step above had used `row["soil_moisture_pct"] != -999.0` instead of checking `>= 0`. Would this catch every future fault code, such as a sensor that reports `-1.0`? Explain why relying on one exact fault-code value is fragile, and propose a more robust preprocessing rule.
2. The Transformation step drew the line between “dry” and “moist” at exactly 20.0%. Modify the `categorize_moisture` function to use three categories (“dry,” “moderate,” “moist”) with thresholds of your choosing, re-run the average-yield comparison by hand for the transformed data given in this section, and describe whether the three-category view changes your interpretation.
3. The Selection stage dropped `rainfall_mm` and `temperature_c`. Suppose the agronomist later suspects that rainfall, not soil moisture, is the real driver of yield, and that soil moisture is only a downstream effect of rainfall. Which KDD stage would you have to revisit to investigate this, and what would you change?
4. Using the *raw*, uncleaned data (before Stage 2), compute the average yield across all 8 records, including the two problematic sensor readings, and compare it to the cleaned Stage-4 averages. Discuss, in a few sentences, what this comparison demonstrates about the cost of skipping Preprocessing.
5. Find (or construct) a small dataset from a domain you are interested in — business, healthcare, education, agriculture, finance, or environmental monitoring — with at least one data-quality problem (a missing value, an obviously invalid value, or a duplicate record). Describe, stage by stage, what Selection, Preprocessing, and Transformation would need to do before that dataset could be meaningfully mined.

5 Real-World Applications of KDD and Data Mining

KDD is not a purely academic exercise; it underlies decisions that affect real people every day. This section surveys a few well-documented applications across different domains, and — just as importantly for a graduate course — examines one widely repeated anecdote that turns out not to hold up under scrutiny.

5.1 Retail and Marketing: Predicting Behavior from Purchase Patterns

Real-World Case Study

In 2012, journalist Charles Duhigg reported in *The New York Times Magazine* on how the retailer Target used purchase-history data mining to identify likely-pregnant shoppers. [4] A Target statistician, Andrew Pole, analyzed purchase histories linked to the store’s baby-registry sign-ups and identified roughly two dozen products — unscented lotion, cotton balls, and supplements such as calcium, magnesium, and zinc, among others — that, in combination, correlated with pregnancy and could be used to estimate a shopper’s approximate due date. Target used this “pregnancy prediction” score to time coupon mailings to specific trimesters. The story became widely known through an anecdote about a father who complained to a Target store after his teenage daughter received baby-product coupons, only to learn she was, in fact, pregnant. Unlike the retail anecdote discussed below, the core facts of the Target case are well documented and were not disputed by the company.

This example illustrates several KDD stages at once: Selection (choosing which purchases to analyze), Transformation (building a composite “pregnancy score” feature from many individual purchases), Data Mining (finding the product combinations that correlated with the registry signal), and Interpretation (deciding how, and how aggressively, to act on the score) — along with the ethical questions that Interpretation must also weigh, since a statistically valid pattern is not automatically an appropriate one to act on.

5.2 Finance: Real-Time Fraud Detection

Real-World Case Study

FICO’s Falcon Fraud Manager, first introduced in the early 1990s, is one of the most widely deployed data mining systems in the world: [REFERENCE NEEDED] it screens a large share of global payment-card transactions in real time, scoring each one for the likelihood of fraud using models trained on historical transaction patterns. According to industry reporting, Falcon-based screening covers roughly two-thirds of all payment cards worldwide and a majority of cards in the United States and United Kingdom, and FICO has stated that it has contributed to major reductions in card-fraud losses since its introduction.

Falcon illustrates why Data Mining (Section 3) is often the *fastest* stage rather than the most important one: scoring a single transaction happens in a fraction of a second, but the system depends on continuous Preprocessing and Transformation of incoming transaction streams, and on an Interpretation and Evaluation process that periodically retrains and re-validates the model as fraud patterns evolve. A fraud-detection model that is never re-evaluated becomes progressively less useful as fraudsters adapt to it — a concrete illustration of why the KDD process is drawn as a cycle rather than a one-time project.

5.3 Healthcare: Early Warning Systems

Real-World Case Study

Johns Hopkins Hospital’s TREWS (Targeted Real-time Early Warning System) is a machine-learning-based tool that monitors electronic health record data to flag patients at risk of sepsis, a life-threatening condition where early treatment substantially improves outcomes. A prospective study published in *Nature Medicine* in 2022 followed roughly 590,000 patients across five Johns Hopkins-affiliated hospitals and identified nearly 6,900 confirmed sepsis cases. [5] The study found that patients whose TREWS alert was confirmed by a clinician within three hours had a meaningfully lower risk of in-hospital death (an adjusted absolute mortality reduction of a few percentage points) compared to patients whose alert was confirmed later. This case highlights a stage of the KDD process that is easy to overlook: even a well-validated model (Data Mining) produces no benefit unless clinicians actually act on its output in time, which is ultimately a question about Interpretation and Evaluation in a real deployment setting — not just about the model’s accuracy on a held-out test set. This course’s companion reading on Python for data science and visualization returns to a related theme: descriptive statistics can mislead if divorced from how a result will actually be used.

5.4 Other Domains

The same process appears, with different data and different stakes, across many other fields. In **agriculture**, the sensor-based moisture/yield example in Section 4 of this reading is a simplified version of *precision agriculture*, where farms mine soil, weather, and yield data to guide irrigation and fertilization decisions at the level of individual fields. In **education**, institutions mine enrollment, grade, and engagement data to identify students at risk of failing or withdrawing from a course — structurally the same kind of early-warning problem as the TREWS example above, applied to a different domain. In **environmental monitoring**, agencies mine sensor networks (air quality, water quality, seismic activity) to detect anomalies and trends that would be impossible to notice by manually reviewing raw readings.

5.5 A Cautionary Note: Not Every Famous Story Is True

Common Pitfall

One of the most frequently repeated data mining anecdotes is the claim that Walmart discovered, through market-basket analysis, that beer and diapers were purchased together (supposedly by young fathers on their way home from work) and responded by placing the two products near each other in stores. This version of the story is **apocryphal**. [REFERENCE NEEDED] Its actual origin traces to a 1992 analysis presented by Thomas Blischok, then of a consulting group associated with Teradata/NCR, who examined roughly 1.2 million market baskets from **Osco Drug** stores — not Walmart. The genuine finding was far more modest than the legend: a time-of-day correlation between beer and diaper purchases in the early evening hours, with no documented age or gender pattern, and no evidence that any retailer physically relocated products because of it. Walmart has never confirmed running such an analysis, and the embellished version of the story appears to be a later media and industry-conference accretion rather than a documented result.

This is worth dwelling on precisely because the story is so appealing: it is a clean, memorable illustration of association-rule mining, which is exactly why it has been repeated in textbooks, conference talks, and vendor marketing for three decades with little scrutiny. As a graduate student in this field, you should hold vendor case studies, conference anecdotes, and even textbook examples (including some in this course’s own readings) to the same standard you would apply to any other claim: what was actually measured, on what data, and is the source checkable?

Practical Exercises

1. For each of the three verified case studies in this section (Target, FICO Falcon, TREWS), identify which KDD stage (Selection, Preprocessing, Transformation, Data Mining, or Interpretation/Evaluation) seems to have required the most ongoing human effort, and justify your answer.
2. The Target case raises an ethical question that a technically correct model does not resolve on its own: a statistically valid prediction (pregnancy likelihood) was used in a way that surprised and unsettled the person it was made about. Propose one process change — at any KDD stage — that could have reduced that risk without necessarily discarding the underlying pattern.
3. Find one additional real-world data mining application (from a news article, a company’s public case study, or an academic paper) in a domain not already covered in this section. Summarize it in 3–5 sentences, and identify which KDD stage the source material focuses on the most (many public case studies over-emphasize Data Mining and say very little about Selection, Preprocessing, or Transformation — see if that pattern holds for the one you found).
4. The beer-and-diapers story is often used specifically because it is memorable, not because it is well documented. Find one other commonly repeated “data science” or “big data” anecdote (from any source) and briefly investigate whether it holds up: what is the original source, and does the popular telling match it?
5. Two of this section’s case studies (FICO Falcon and TREWS) depend on a model continuing to work well *after* deployment, not just on its initial validation. For one of them, describe one concrete way its performance could degrade over time, and which KDD stage would

need to be revisited to catch and fix that degradation.

6 Python Basics: Variables, Data Types, and Operators

Every stage of the KDD process in this reading was implemented in Python. The remainder of this reading reviews the language foundations needed to read and write that code confidently, starting from the smallest building blocks. If you already have programming experience, treat this as a fast, example-driven refresher rather than a first introduction; if Python is new to you, work through every example by running it yourself.

6.1 Variables and Basic Data Types

A **variable** is a name bound to a value. [6] Python is **dynamically typed**: you do not declare a variable's type in advance, and the same name can be rebound to a value of a different type later (though doing so on purpose, rather than by accident, is a good habit). The built-in types you will use constantly in this course are `int` (whole numbers), `float` (decimal numbers), `str` (text), `bool` (True/False), and `None` (the deliberate absence of a value — as already seen with the missing sensor reading in Section 4).

```
patient_id = "P-10432"      # str
age = 47                    # int
body_temp_c = 38.6          # float
is_fever = body_temp_c > 37.5 # bool, derived from a comparison
allergy_notes = None        # NoneType -- "not yet recorded", not "none"
```

```
patient_id = 'P-10432' -> type str
age = 47 -> type int
body_temp_c = 38.6 -> type float
is_fever = True -> type bool
allergy_notes = None -> type NoneType
```

Common Pitfall

`None` is not the same as 0, `False`, or an empty string `""`. It specifically means “no value is recorded here.” Confusing a genuinely missing value with a zero value is a common and consequential data-quality bug — in the farm-sensor example, treating the missing reading as 0.0 instead of `None` would have silently invented a very dry field that was never actually measured.

6.2 Arithmetic Operators

Python supports the arithmetic operators you would expect, plus two that surprise newcomers: floor division (`//`) and modulo (`%`).

```
visit_fee = 120.00
lab_fee = 45.50
discount_pct = 10

subtotal = visit_fee + lab_fee
```

```
discount_amount = subtotal * discount_pct / 100
total_due = subtotal - discount_amount
```

```
subtotal = visit_fee + lab_fee = 165.5
discount_amount = subtotal * discount_pct / 100 = 16.55
total_due = subtotal - discount_amount = 148.95
integer division 47 // 10 = 4    (whole visits per decade of age, contrived)
modulo 47 % 10 = 7    (remainder)
exponent 2 ** 5 = 32
```

6.3 Comparison, Logical, and Assignment Operators

Comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) always produce a `bool`. Logical operators (`and`, `or`, `not`) combine boolean values — these will become the building blocks of every filtering step in the rest of this course, including the Preprocessing filters used in Section 4.

```
temperature_threshold = 37.5
age_threshold = 65

is_high_risk = (body_temp_c > temperature_threshold) and (age >= age_threshold)
is_worth_flagging = (body_temp_c > temperature_threshold) or (age >= age_threshold)
```

```
body_temp_c > 37.5    -> True
age >= 65             -> False
high risk (AND)       -> False
worth flagging (OR)   -> True
NOT is_fever          -> False
age != 47             -> False
```

Assignment operators such as `+=` and `-=` update a variable in place: `running_total += subtotal` is shorthand for `running_total = running_total + subtotal`. This shorthand is used constantly when accumulating totals inside a loop, as Section 7 and the capstone in Section 12 both do.

Tip

A single equals sign (`=`) is assignment; a double equals sign (`==`) is comparison. Writing `if age = 65:` instead of `if age == 65:` is one of the most common early Python mistakes — and helpfully, Python itself refuses to run code that makes this particular error, raising a `SyntaxError` rather than silently doing the wrong thing.

Practical Exercises

1. Write three variables representing a single retail transaction: `item_price` (float), `quantity` (int), and `is_member_discount_applied` (bool). Then write one expression that computes the final total, applying a 5% discount only when `is_member_discount_applied` is `True`.
2. Explain, in your own words, why `7 / 2` and `7 // 2` produce different results in Python, and give a realistic scenario (in any of this course's suggested domains) where you would want floor division instead of ordinary division.
3. Given `score_a = 82` and `score_b = 82.0`, are these two values equal under Python's `==` operator? Are they the same *type*? Run both checks and explain the distinction between

value equality and type identity.

4. A dataset field is supposed to record a patient's age but was accidentally populated with 0 for every patient whose age was not collected, instead of using `None`. Write a short explanation (2–3 sentences, no code required) of why this design choice could distort a later average-age calculation, connecting your answer to the Preprocessing stage from Section 4.
5. Write and evaluate a compound boolean expression, using `and/or/not`, that flags an environmental sensor reading as “needs review” when its temperature is below -10 or above 45 degrees Celsius, *or* its humidity reading is missing (`None`). Test it against at least four different hand-constructed readings and explain each result.

7 Control Flow: Conditional Statements and Loops

Control flow is what turns individual expressions into a program: it decides which code runs, and how many times. Every filtering, cleaning, and aggregation step in Section 4 relied on exactly the constructs introduced here.

7.1 Conditional Statements

An `if/elif/else` chain lets a program branch based on a condition. Conditions are evaluated top to bottom, and the first one that is `True` wins — later conditions are never even checked.

```
def letter_grade(score):  
    if score >= 90:  
        return "A"  
    elif score >= 80:  
        return "B"  
    elif score >= 70:  
        return "C"  
    elif score >= 60:  
        return "D"  
    else:  
        return "F"
```

```
score 95 -> grade A  
score 82 -> grade B  
score 71 -> grade C  
score 58 -> grade F
```

Common Pitfall

Order matters in an `if/elif` chain. If the conditions above were reordered so that `score >= 60` came first, every passing score — including an actual 95 — would be graded “D,” because the chain stops at the first `True` condition it reaches. When several conditions could be true simultaneously, always order them from most specific to least specific.

7.2 for Loops

A `for` loop repeats a block of code once for each item in a sequence — a list, a string, a range of numbers, or (as you will see throughout this course) rows of a dataset.

```
scores = [95, 82, 71, 58, 88, 64, 90]

for score in scores:
    print(f"{score} -> {letter_grade(score)}")
```

```
95 -> A
82 -> B
71 -> C
58 -> F
88 -> B
64 -> D
90 -> A
```

When you need both the value and its position, `enumerate()` avoids the error-prone pattern of manually tracking an index variable:

```
for i, score in enumerate(scores, start=1):
    print(f"student #{i}: score {score}")
```

7.3 while Loops

A `while` loop repeats as long as a condition remains true, which makes it the right tool when you do not know in advance how many iterations will be needed — for example, polling a sensor feed until a stopping condition is met.

```
consecutive_stable = 0
attempts = 0
readings = []
while consecutive_stable < 3 and attempts < 20:
    value = read_sensor()
    readings.append(value)
    attempts += 1
    if 30.0 <= value <= 70.0:
        consecutive_stable += 1
    else:
        consecutive_stable = 0
```

```
readings taken: [45.2, 56.0, 92.4, 46.6, 50.8, 58.7]
attempts used: 6, stopped because 3 consecutive stable readings
```

Common Pitfall

Every `while` loop needs a condition that is guaranteed to eventually become false, or a hard cap like the `attempts < 20` check above. Forgetting this produces an **infinite loop** — one of the most common bugs for programmers new to `while` loops, and one reason many experienced programmers prefer a `for` loop with a known bound whenever one will do the job.

7.4 List Comprehensions

A **list comprehension** builds a new list from an existing sequence in a single, readable expression, combining what would otherwise be a `for` loop plus an `if` statement plus repeated calls to `.append()`.

```
grades = [letter_grade(s) for s in scores]
passing_scores = [s for s in scores if s >= 60]
curved_passing = [s + 5 for s in scores if s >= 60]
```

```
grades = ['A', 'B', 'C', 'F', 'B', 'D', 'A']
passing_scores = [95, 82, 71, 88, 64, 90]
curved_passing = [100, 87, 76, 93, 69, 95]
```

Tip

List comprehensions are exactly what the Selection, Preprocessing, and Transformation steps in Section 4 used under the hood (`selected = [select_fields(row) for row in raw_log]`, and similarly for `cleaned` and `transformed`). Recognizing this pattern — `[expression for item in sequence if condition]` — will make a large fraction of the data-preparation code you write in this course easier to both write and read.

Practical Exercises

1. Write a function `risk_category(days_since_last_login)` that returns "low" for 0–3 days, "medium" for 4–10 days, and "high" for more than 10 days, using an `if/elif/else` chain. Test it on at least five values, including a boundary value like exactly 10.
2. Using a `for` loop (not a list comprehension), write code that counts how many values in a list of exam scores fall into each letter grade from `letter_grade()`, storing the counts in a dictionary. (This previews the dictionary-based counting pattern used again in Section 9 and the Section 12 capstone.)
3. Rewrite the `while` loop example above so that instead of stopping after 3 consecutive “stable” readings, it stops after 3 consecutive readings *outside* the safe range (a simple anomaly-detection stopping rule). Explain, in a sentence, what real-world monitoring scenario this modified rule would be useful for.
4. Convert the following `for` loop into an equivalent list comprehension, and explain in one sentence why the comprehension version is or is not more readable: `result = []; for x in readings: if x > 50: result.append(x * 2)`.
5. Using the farm-sensor `transformed` data from Section 4 (or reconstructing it from the raw log given there), write a list comprehension that extracts just the `field_id` values of every field categorized as "moist". Then write a second version using a `for` loop instead, and confirm both produce the same result.

8 Functions in Python

A **function** packages a block of code under a name, so it can be reused without being retyped, and so that a complex process can be described as a sequence of clearly named steps — exactly how Section 4 broke the KDD process into `select_fields()`, `categorize_moisture()`, and `average()`.

8.1 Defining and Calling a Function

```
def celsius_to_fahrenheit(temp_c):  
    """Convert a Celsius temperature to Fahrenheit."""  
    return temp_c * 9 / 5 + 32
```

```
0 C -> 32.0 F  
20 C -> 68.0 F  
37 C -> 98.6 F  
100 C -> 212.0 F
```

The text in triple quotes immediately after the `def` line is a **docstring**: a short description of what the function does, aimed at the next person (often, a few months later, yourself) who has to read it without re-deriving the logic from scratch.

8.2 Parameters, Return Values, and Default Arguments

A function can take multiple parameters, and any parameter can be given a **default value**, making it optional at the call site.

```
def loan_monthly_payment(principal, annual_rate_pct, years, periods_per_year=12):  
    """Compute a fixed payment for a standard amortizing loan."""  
    n_periods = years * periods_per_year  
    rate_per_period = (annual_rate_pct / 100) / periods_per_year  
    if rate_per_period == 0:  
        return principal / n_periods  
    payment = principal * (rate_per_period * (1 + rate_per_period) ** n_periods) / \  
        ((1 + rate_per_period) ** n_periods - 1)  
    return payment
```

```
loan_monthly_payment(20000, 6.0, 5) -> 386.66  
loan_monthly_payment(20000, 6.0, 5, periods_per_year=4) -> 1164.91
```

The first call relies entirely on the default (`periods_per_year=12`, i.e., monthly payments); the second overrides it explicitly to compute quarterly payments instead. Naming the argument (`periods_per_year=4`) rather than passing 4 positionally makes the call self-documenting — a habit worth building early, especially for functions with several parameters.

8.3 Returning Multiple Values

A Python function can return more than one value at once, which Python packages as a tuple (Section 9.2) that can be unpacked directly at the call site.

```
def summarize(values):  
    """Return (minimum, maximum, mean) of a list of numbers, computed  
    without any external library."""  
    total = 0  
    lowest = values[0]  
    highest = values[0]  
    for v in values:  
        total += v  
        if v < lowest:
```

```

        lowest = v
    if v > highest:
        highest = v
    mean = total / len(values)
    return lowest, highest, mean

class_sizes = [22, 31, 18, 27, 25, 40, 19]
low, high, mean = summarize(class_sizes)

```

```

class_sizes = [22, 31, 18, 27, 25, 40, 19]
summarize(class_sizes) -> min=18, max=40, mean=26.00

```

Key Idea

`summarize()` computes a minimum, maximum, and mean by hand, using nothing but a loop and a few comparisons. This is deliberate: seeing the computation spelled out once, in plain Python, is what will make the vectorized NumPy and Pandas equivalents in the next reading (`values.min()`, `values.max()`, `values.mean()`) meaningful rather than magical — they compute exactly this, just faster and over much larger arrays.

8.4 Function Composition

Functions can call other functions, letting you build complex behavior from small, independently understandable pieces.

```

def flag_oversized_classes(class_sizes, cap=30):
    """Return the oversized class sizes, and by how much the largest
    class exceeds the cap, reusing summarize()."""
    _, largest, _ = summarize(class_sizes)
    oversized = [size for size in class_sizes if size > cap]
    over_by = largest - cap if largest > cap else 0
    return oversized, over_by

```

```

flag_oversized_classes(class_sizes, cap=30) -> oversized=[31, 40], over_by=10

```

Notice the underscore (`_`) used when unpacking `summarize()`'s return value: this is a common Python convention meaning “I need this position's value, but I am not going to use it,” here discarding the minimum and mean since `flag_oversized_classes` only needs the maximum.

Practical Exercises

1. Write a function `fahrenheit_to_celsius(temp_f)` that reverses `celsius_to_fahrenheit()`, and verify it round-trips correctly (i.e., converting 37°C to Fahrenheit and back returns 37.0) for at least three test values.
2. Modify `loan_monthly_payment` to accept a new optional parameter, `extra_payment=0`, representing a fixed extra amount paid each period on top of the computed payment, and have the function return both the total payment per period *and* the number of periods interest is charged on, assuming the extra payment reduces principal faster. (You do not need to implement full amortization; a documented simplifying assumption is acceptable and expected at this stage — state it clearly.)

3. Write a function `summarize_by_category(records, category_key, value_key)` that takes a list of dictionaries (like the transformed farm-sensor or bookstore records from Sections 4 and 12) and returns a dictionary mapping each distinct category to the average of `value_key` within that category, using only loops and the `summarize()`-style pattern from this section (no external libraries).
4. Explain, using the `flag_oversized_classes` example, why the parameter `cap=30` being a default rather than a required argument is a reasonable design choice for this function, and describe a situation where you *would* want to force the caller to specify a cap explicitly instead.
5. Write a short function that takes a raw record (a dictionary) from the bookstore scenario in Section 12 and returns `True` if the record should be *kept* during Preprocessing (i.e., it encapsulates the validity check as its own named function, rather than an inline condition). Explain why extracting this into a named function is good practice even though it is used only once in the current code.

9 Basic Data Structures: Lists, Tuples, Dictionaries, and Sets

Python's four built-in collection types cover almost every data-representation need you will encounter before this course introduces NumPy arrays and Pandas DataFrames. Choosing the right one for a given task is itself an important skill: each one makes a different trade-off between ordering, mutability, and uniqueness.

9.1 Lists: Ordered and Mutable

A **list** holds an ordered, changeable sequence of items, accessed by position.

```
daily_sales = [412.50, 389.00, 455.75, 402.10, 610.25, 720.00, 388.90]

daily_sales.append(399.40)    # mutable: add a new day
```

```
daily_sales[0] (Monday) = 412.5
daily_sales[-1] (Sunday) = 388.9
daily_sales[4:6] (Fri-Sat) = [610.25, 720.0]
after append(399.40): [412.5, 389.0, 455.75, 402.1, 610.25, 720.0, 388.9, 399.4]
len(daily_sales) = 8
sorted(daily_sales) = [388.9, 389.0, 399.4, 402.1, 412.5, 455.75, 610.25, 720.0]
```

Negative indices count from the end (`-1` is the last element), and a `start:stop` slice extracts a sub-list without modifying the original.

9.2 Tuples: Ordered and Immutable

A **tuple** looks like a list but cannot be modified after creation, which makes it a natural fit for values that represent a fixed, meaningful group — like a coordinate pair.

```
station_location = (39.1031, -84.5120)    # a monitoring station's coordinates
lat, lon = station_location              # tuple unpacking
```

```
station_location = (39.1031, -84.512)
attempting station_location[0] = 40.0 raises: 'tuple' object does not support item
assignment
unpacked: lat=39.1031, lon=-84.512
```

Tip

A useful rule of thumb: reach for a tuple when the collection's *length and meaning are fixed* (a latitude/longitude pair is always exactly two numbers, in a fixed order), and reach for a list when items will be added, removed, or reordered over the collection's lifetime.

9.3 Dictionaries: Key-Value Mappings

A **dictionary** maps keys to values, which is the natural representation for a single record with named fields — exactly how every row of the farm-sensor and bookstore datasets in this reading has been represented.

```
patient_record = {
    "patient_id": "P-10432",
    "age": 47,
    "diagnosis_code": "J45.909",
    "visits_this_year": 3,
}
patient_record["visits_this_year"] += 1
patient_record["last_visit_flagged"] = True
```

```
patient_record['age'] = 47
after incrementing visits_this_year: 4
after adding a new key: {'patient_id': 'P-10432', 'age': 47, 'diagnosis_code': 'J45.909',
    'visits_this_year': 4, 'last_visit_flagged': True}
keys: ['patient_id', 'age', 'diagnosis_code', 'visits_this_year', 'last_visit_flagged']
.get('allergy', 'not recorded') = not recorded
```

Common Pitfall

Accessing a missing key with square brackets (`patient_record["allergy"]`) raises a `KeyError` and crashes the program. The `.get(key, default)` method instead returns a fallback value when the key is absent, which is almost always what you want when processing real-world records that may not all have the same fields — as, again, the farm-sensor data in Section 4 demonstrated with its missing readings.

9.4 Sets: Unordered Collections of Unique Elements

A **set** holds unique elements with no guaranteed order, and supports the mathematical set operations: union, intersection, difference, and symmetric difference.

```
section_a_topics = {"loops", "functions", "lists", "dictionaries"}
section_b_topics = {"functions", "dictionaries", "sets", "recursion"}

intersection = section_a_topics & section_b_topics    # topics in both sections
```

```
intersection (both sections) = {'functions', 'dictionaries'}
difference (A only) = {'lists', 'loops'}
symmetric_difference (exactly one section) = {'lists', 'sets', 'recursion', 'loops'}
```

Common Pitfall

Sets are *unordered*: printing the same set in two different program runs can display its elements in a different order (this is a real, observable property of Python sets, not a simplification for this reading). Never rely on set iteration order; if you need a predictable order, convert to a sorted list first with `sorted(my_set)`.

A common, practical use of sets is de-duplication:

```
raw_field_ids = ["A1", "A2", "A3", "A1", "B1", "A2", "B1", "C1"]
unique_field_ids = set(raw_field_ids)
```

```
raw_field_ids = ['A1', 'A2', 'A3', 'A1', 'B1', 'A2', 'B1', 'C1'] (length 8)
duplicates removed: 3
```

Key Idea

Choosing a data structure is itself a design decision with consequences: a list of sensor readings that should never contain duplicates might be better represented as a set from the start, catching a data-entry bug the moment it happens rather than discovering it later during Preprocessing. Part of learning KDD is learning to make these small representational choices deliberately rather than by default.

Practical Exercises

1. Given `weekly_temps = [68, 71, 74, 69, 72, 77, 75]` (degrees Fahrenheit, one week of daily readings), write code using list indexing/slicing to extract: the first three days, the last two days, and the single highest reading (without using a built-in sorting function).
2. Explain why `student_grade = ("A-", 3.7)` is a reasonable use of a tuple, but a *list* of ten such pairs, one per student, is often more appropriately a *list of dictionaries* once each student also needs a name and ID. What does the dictionary give you that the plain tuple does not?
3. Using a dictionary as an accumulator (the same pattern used in `revenue_by_genre` in the Section 12 capstone, and previewed with `summarize_by_category` in Section 8), write code that counts how many times each letter grade ('A' through 'F') appears in a list of at least ten exam scores.
4. Two hospital wards each keep a set of the diagnosis codes they have treated this month. Using set operations, write code to find: diagnosis codes seen at either ward, diagnosis codes seen at both wards, and diagnosis codes seen at the first ward but not the second. Explain, for a hospital administrator, what each of these three results would actually be useful for.
5. A dataset of customer emails contains some exact duplicate entries due to a logging bug. Using a set, write code that reports both the number of duplicate entries removed and the resulting list of unique emails, *sorted alphabetically* so the output is reproducible across

runs (recall the warning above about set ordering).

10 Best Practices and Common Pitfalls

- **Do not skip stages of the KDD process, even informally.** Section 4’s fault-code reading (−999.0) would have silently corrupted the mining step if Preprocessing had been skipped — and this exact failure mode (a numeric sentinel value standing in for a missing reading) recurs constantly in real sensor, survey, and legacy-system data.
- **Treat None as “unknown,” not as zero, false, or empty.** Confusing a missing value with a real zero value is one of the most common and most consequential data-preparation mistakes, as Section 6’s discussion of the farm-sensor data makes concrete.
- **Choose your data structure deliberately.** A dictionary-of-fields naturally represents one record (Section 9.3); a list naturally represents a sequence of records or readings (Section 9.1); a set is the right tool the moment “no duplicates” or “membership only, order does not matter” becomes a requirement (Section 9.4).
- **A pattern found during Data Mining is not yet knowledge.** Every case study and worked example in this reading — the farm-sensor moisture comparison, Target’s pregnancy score, TREWS’s sepsis alerts — required a human Interpretation and Evaluation step before the pattern became something anyone should act on.
- **Be skeptical of vendor case studies and famous anecdotes, including the ones repeated in textbooks.** The beer-and-diapers story in Section 5.5 survived for three decades in course material and marketing precisely because nobody checked it against a primary source.
- **Name your functions and steps after what they do, not how they do it.** Names like `categorize_moisture` and `is_valid` describe intent; a reader does not need to reread the implementation to know what the code is for.

11 A Practical Mental Model

Key Idea

When you are handed a new dataset and a vague goal (“find something useful in this”), work through the five KDD stages in order, out loud if it helps: what, specifically, do I actually need from this data (Selection)? What is wrong with it as given (Preprocessing)? What shape does it need to be in before an algorithm can use it (Transformation)? What is the simplest technique that could reveal a pattern relevant to my question (Data Mining)? And, most importantly: if I find something, what would convince me — and a skeptical domain expert — that it is real and worth acting on (Interpretation and Evaluation)? Every example in this reading, from the eight-row farm-sensor log to the well-documented industry case studies, followed exactly this sequence; only the size of the data and the sophistication of the algorithm changed.

12 Capstone Exercise: An End-to-End Mini Walkthrough

This capstone applies every Python concept from Sections 6–9 to a complete, independent pass through the KDD process, on a new scenario: a small online bookstore’s raw order log, which mixes a missing value, an invalid value, and a genuine analytical question in the same ten rows.

```
raw_orders = [
    {"order_id": "01001", "customer": "Aiko", "genre": "mystery", "price": 14.99, "qty":
      1, "country": "Japan"},
    {"order_id": "01002", "customer": "Bilal", "genre": "sci-fi", "price": 9.50, "qty"
      : 2, "country": "UAE"},
    {"order_id": "01003", "customer": "Chen", "genre": "mystery", "price": 14.99, "qty"
      : 1, "country": "China"},
    {"order_id": "01004", "customer": "Dana", "genre": None, "price": 22.00, "qty"
      : 1, "country": "USA"}, # missing genre
    {"order_id": "01005", "customer": "Aiko", "genre": "sci-fi", "price": 9.50, "qty"
      : 3, "country": "Japan"},
    {"order_id": "01006", "customer": "Elin", "genre": "romance", "price": -5.00, "qty"
      : 1, "country": "Sweden"}, # invalid price
    {"order_id": "01007", "customer": "Farid", "genre": "mystery", "price": 14.99, "qty"
      : 2, "country": "Egypt"},
    {"order_id": "01008", "customer": "Grace", "genre": "romance", "price": 11.25, "qty"
      : 1, "country": "USA"},
    {"order_id": "01009", "customer": "Chen", "genre": "sci-fi", "price": 9.50, "qty"
      : 1, "country": "China"},
    {"order_id": "01010", "customer": "Hana", "genre": "romance", "price": 11.25, "qty"
      : 2, "country": "Egypt"},
]
```

The business question: which genre generates the most revenue? **Selection** keeps only the fields that question needs (customer identity is dropped); **Preprocessing** removes the missing-genre and negative-price records; **Transformation** derives an `order_total` and a `revenue_tier` category; **Data Mining** aggregates revenue and order count by genre using a dictionary accumulator; **Interpretation** draws a conclusion with appropriate caveats.

```
revenue_by_genre = {}
count_by_genre = {}
for o in transformed_orders:
    genre = o["genre"]
    revenue_by_genre[genre] = revenue_by_genre.get(genre, 0) + o["order_total"]
    count_by_genre[genre] = count_by_genre.get(genre, 0) + 1
```

```
mystery : 3 order(s), total revenue $59.96, avg $19.99/order
romance  : 2 order(s), total revenue $33.75, avg $16.88/order
sci-fi   : 3 order(s), total revenue $57.00, avg $19.00/order
```

Distinct countries per genre (using sets to find unique values):

```
mystery : ['China', 'Egypt', 'Japan']
romance  : ['Egypt', 'USA']
sci-fi   : ['China', 'Japan', 'UAE']
```

`'mystery'` generated the most total revenue (\$59.96) across 3 orders `in` this sample.
Caveats: only 10 raw orders were available, 2 were dropped as invalid

during Preprocessing, and `mystery`'s total is driven by a small number of higher-priced orders rather than broad popularity -- exactly the kind of judgment call that belongs to a human at the Interpretation stage, not to the aggregation step itself.

Notice that the “distinct countries per genre” computation reuses `set()` from Section 9.4 to answer a question — *does this genre sell in more than one country?* — that the dictionary accumulator alone could not answer.

Practical Exercises

1. Reproduce this entire walkthrough yourself from the raw data given above, without copying the accumulator code verbatim: write your own `select_fields`, `is_valid`, and `revenue_tier` functions, and confirm your output matches the listing above exactly.
2. The current analysis treats `qty` correctly when computing `order_total` but never examines it on its own. Extend the Data Mining stage to also report total *units sold* (summed `qty`, not revenue) per genre, and discuss whether the genre with the highest revenue is also the genre with the highest unit sales — and if not, what that discrepancy might mean for a bookstore's purchasing decisions.
3. The Preprocessing step drops any order with a missing genre or a negative price. Propose and implement an alternative Preprocessing policy for the missing-genre case that does *not* discard the order entirely (for example, imputing a placeholder category), and discuss one risk this alternative policy introduces that the drop-it policy avoided.
4. Add two new orders of your own construction, in a genre not currently present in the data (e.g., “fantasy”), and re-run the full pipeline. Does the identity of the highest-revenue genre change? Use this to discuss, concretely, why a sample of 10–12 orders is too small to support a confident business decision.
5. This walkthrough and the Section 4 farm-sensor walkthrough both follow the same five-stage structure but analyze completely different domains. Write a short paragraph (5–8 sentences) comparing the two: what stayed the same about the *process*, and what had to change because of the *data and the question*?

13 Summary

This reading introduced knowledge discovery in databases (KDD) as an end-to-end process for turning large, messy collections of data into knowledge people can act on, distinguishing it from the narrower “data mining” stage nested inside it, and from the related but distinct fields of statistics, database systems, machine learning, and artificial intelligence. The classic KDD process (Selection, Preprocessing, Transformation, Data Mining, Interpretation and Evaluation) and its industry-standard cousin, CRISP-DM, describe the same underlying work with different emphases, and both are explicitly iterative rather than linear. A worked example — a small, deliberately messy farm-sensor dataset — showed what each stage concretely contributes, and why skipping Preprocessing in particular can silently corrupt every stage that follows. Real-world applications from retail, finance, and healthcare demonstrated that this same process underlies systems used at enormous scale, while a close look at the widely repeated “beer and diapers” anecdote demonstrated that even well-known data mining stories deserve the same scrutiny as any other claim. The second half of this reading reviewed the Python foundations — variables and data types, arithmetic/-comparison/logical/assignment operators, conditional statements, loops and list comprehensions,

functions, and the four built-in collection types — that every later reading in this course, and the KDD process itself, depends on. The capstone walkthrough tied both halves together by carrying a new dataset through the full KDD process using nothing but the Python constructs just reviewed.

14 Discussion Questions

1. Section 2.1 argues that “data mining” and “KDD” are used almost interchangeably in casual usage despite naming different things. Does this looseness in terminology cause real problems in practice, or is it mostly harmless? Justify your position with an example.
2. Section 3 presents the classic KDD process and CRISP-DM as two descriptions of “essentially the same underlying work.” Identify one situation in which the choice between them would actually matter — where following one model rather than the other would lead a team to do something differently, not just describe it differently.
3. Section 4’s farm-sensor example found a large gap in average yield between “dry” and “moist” fields using only 6 cleaned records. What is the difference between a pattern being *real* (it would replicate on new data) and a pattern being *present* (it is true of this specific sample)? Which one did the worked example actually establish?
4. Section 5’s Target case study and its beer-and-diapers case study are both about retail data mining, but the reading treats them very differently. What specifically distinguishes a well-documented case from an apocryphal one, and what would you personally require before repeating a data-mining anecdote you encountered secondhand?
5. Section 9.4 warns that set iteration order is not guaranteed. Beyond the specific example given, what does this suggest about the broader risk of writing code (or, later in this course, an analysis) that happens to work correctly by accident, due to an implementation detail that is not actually guaranteed to hold?
6. The Section 12 capstone found that adding two new orders in a new genre could change which genre appears most profitable. What sample size would you consider sufficient before trusting a similar genre-revenue conclusion in an actual small business? What would you want to know about *how* the sample was collected, beyond just its size?

15 Key Takeaways for CIS 635

1. **KDD is a process, not an algorithm.** Data Mining is one of five classic stages, and in every example in this reading — the farm-sensor walkthrough, the bookstore capstone, and the industry case studies — the surrounding stages (especially Preprocessing and Interpretation) demanded at least as much attention as the mining step itself.
2. **The classic KDD process and CRISP-DM describe the same work with different emphases.** CRISP-DM’s explicit Business Understanding and Deployment phases make visible two concerns the classic five-stage process leaves implicit but does not ignore.
3. **Feedback loops are not an edge case — they are the normal case.** Every process diagram in this reading is drawn with loops back to earlier stages, because a poor result at any stage routinely means an earlier stage needs revisiting.

4. **A pattern is not knowledge until a human evaluates it.** Section 4’s yield comparison, Section 5’s three verified case studies, and the Section 12 capstone all reached a numeric result before reaching a conclusion — and the gap between the two is exactly the work of the Interpretation and Evaluation stage.
5. **Widely repeated data mining stories are not automatically true.** The beer-and-diapers anecdote misattributed a real, modest 1992 finding about Osco Drug to Walmart with an invented narrative — a caution worth carrying into every case study, textbook example, and vendor claim you encounter in this field, including in this course’s own readings.
6. **Python’s built-in data structures already cover most of what KDD’s early stages need.** Lists, tuples, dictionaries, and sets — with no external library required — were sufficient to implement Selection, Preprocessing, Transformation, and simple Data Mining in every example in this reading.
7. **Writing the computation out by hand once makes the library version meaningful later.** The `average()` and `summarize()` functions in Sections 4 and 8 compute, step by step, exactly what NumPy’s and Pandas’ built-in statistics methods compute in the next reading — just without the speed or the syntax sugar.

References

- [1] Hasan, K. (2026). *Data Mining and Knowledge Discovery*. Grand Valley State University. https://github.com/mdkamrulhasan/data_mining_kdd
- [2] Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From data mining to knowledge discovery in databases. *AI Magazine*, 17(3), 37–54. <https://doi.org/10.1609/aimag.v17i3.1230>
- [3] Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.
- [4] Duhigg, C. (2012, February 16). How companies learn your secrets. *The New York Times Magazine*.
- [5] Adams, R., Henry, K. E., Sridharan, A., Soleimani, H., Zhan, A., Rawat, N., et al. (2022). Prospective, multi-site study of patient outcomes after implementation of the TREWS machine learning-based early warning system for sepsis. *Nature Medicine*, 28, 1455–1460. <https://doi.org/10.1038/s41591-022-01894-0>
- [6] Python Software Foundation. (n.d.). *The Python tutorial*. <https://docs.python.org/3/tutorial/>